

PPTGO SDK 接入指引

概述

PPTGO SDK 作为开放编辑器形态，供第三方接入后以实现一致的产品形态。PPTGO SDK 只提供核心的白板协同编辑能力，不参与权限管理、用户身份相关功能流程，相关文件管理及权限机制等由接入方业务自行决定。PPTGO SDK 只针对接入方 这个企业授予编辑器使用权限，在 PPTGO SDK 看来所创建的文件归属于 接入方 这个企业。PPTGO SDK 通过 OAuth2.0 的 [客户端凭证方式](#) 来进行 接入方 的身份验证和授权。

接入开发点

- 访问授权流程及相关OpenAPI接入（详情见 [OpenAPI](#)）
- 前端开发pptgo文件编辑页，嵌入PPTGO SDK编辑器（iframe嵌入方式，编辑器授权方式提供token回调和访问代理两种方式）
- 开发回调通知接口，供PPTGO SDK 通过回调给到相关事件通知（可选）

Host

1. 生产环境

<https://sdk.pptgo.cn/pptsdk>

2. 私有化部署

以实际部署环境为准

3. 沙箱环境

<https://sdk-pre.pptgo.cn/pptsdk>

下文中的{Prefix}变量均表示对应环境域名

请求规范说明

- 网络协议：HTTPS 协议，私有化以实际部署为准
- 编码格式：UTF-8 编码。
- 请求方法：GET 、 POST 、 PUT 、 PATCH 、 DELETE （具体方法见接口描述）。
- 请求参数：支持 3 种，分别为 Path Parameters （路径参数）、Query Parameters （查询参数）和 Body Parameters （请求体参数）。
- 请求头部：需要使用 Authorization 携带 AccessToken ，用于后台的认证与鉴权。
- 权限说明：接口描述会标注 AccessToken 需要的 scope ，而 scope 则依赖集成方的所获得的 scope 和发起授权时指定的 scope ，scope 支持通配符 * （例如 file.* 表示包含 file.create 、 file.delete 、 file.copy 等以 file. 开头的权限）。
- 响应结果：JSON 数据格式。API 的响应体结构包含 code、msg、data 三个部分，有的只包含 code、msg 。code 为返回码，msg 为返回码描述，data 为调用 API 后返回的结果；如果 code 不为 0 或 200 则认为请求失败，失败信息可参考 msg 。

注：msg 仅作为参考，后续可能会有变动，不可作为是否调用成功的判断依据。

接口描述说明

接口将按照以下模板进行描述说明。

接口说明：
.....
请求格式：
.....
Scope：
.....
请求头部：
.....
请求参数：
.....
请求体：
.....
响应体：
.....
请求示例：
.....
响应示例：
.....

快速开始

示例一：创建空白文档进行编辑

- 1. 获取client_id 、 client_secret
见 [获取client_id 、 client_secret](#获取client_id 、 client_secret)
- 2. 申请应用access_token
【服务端】 根据接口文档 [获取应用级access_token](#2. 获取应用级access_token) 获取访问令牌

请求示例

```
curl --location --request POST '{Prefix}/oauth/token' \  
--form 'grant_type="client_credentials"' \  
--form 'scope="all_scopes"' \  
--form 'client_id="{client_id}"' \  
--form 'client_secret="{client_secret}"'
```

响应示例

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJjaWQiOiJpSW1sdGZLY295WnBuUzRBIiwiaWZlbnkiOiMTIiLCJpc3MiOiJib2FyZG1peC1zZGsiLCJqdGkiOiI1ZmYyMjY1Ni1lNDQ3LTQ1NmYtYWUyMS02ZWV3NDI3YTVjYzgiLCJpYXQiOiJlNDQ3LTQ1NmYtYWUyMS02ZWV3NDQ4MSwic2NwIjpbImFsbF9zY29wZXMiXX0.KcDW_eVW0l4jaf0NxU6qyKs7ZDZxAVjuauaWYyskgJk",
  "expires_in": 1799,
  "scope": "all_scopes",
  "token_type": "bearer"
}
```

在后续OpenAPI调用需根据token_type和access_token添加Authorization授权访问头。

示例：Authorization: Bearer

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJjaWQiOiJpSW1sdGZLY295WnBuUzRBIiwiaWZlbnkiOiMTIiLCJpc3MiOiJib2FyZG1peC1zZGsiLCJqdGkiOiI1ZmYyMjY1Ni1lNDQ3LTQ1NmYtYWUyMS02ZWV3NDI3YTVjYzgiLCJpYXQiOiJlNDQ3LTQ1NmYtYWUyMS02ZWV3NDQ4MSwic2NwIjpbImFsbF9zY29wZXMiXX0.KcDW_eVW0l4jaf0NxU6qyKs7ZDZxAVjuauaWYyskgJk
```

3. 新建空白文档

【服务端】调用 [新建文件](#) OpenAPI接口创建一个空白文件，调用成功后会返回创建的文件信息，示例如下：

```
{
  "code": 0,
  "msg": "code-0",
  "data": {
    "file_key": "0olokjVRXrdokufDtaFEJA",
    "created_at": 1661758256,
    "updated_at": 1661758256
  }
}
```

在响应体中的 data.file_key 为 PPTGO SDK 对每个文件的唯一标识，接入方 需保存该file_key，在调用 PPTGO SDK 编辑器打开文件时，需指定 file_key 来打开对应文件。

4. 前端接入编辑器

【接入方前端】在index.html中通过script标签引入pptgo-sdk.js依赖，会在window上挂载PPTGOSdk

```
<script src="{Prefix}/app/pptgo-sdk.js" ></script>
```

【接入方前端】继承PPTGOSdk并实现必须的三个抽象方法

```

class SdkBoxHelper extends PPTGOSdk {
  protected override async getAccessToken(): Promise<{
    token: string;
    expiresIn: string
  }> {
    const res = await fetch(`/api/sdk/oauth/token?file_key=${ FILE_KEY }&role=${
FILE_ROLE }`);
    const data = (await res.json()).data;
    return {
      token: data.access_token,
      expiresIn: data.expires_in
    }
  }

  protected override getFileInfo(): Promise<{
    name: string;
    fileKey: string;
    isEditor: boolean;
  }> {
    return Promise.resolve({
      name: "文件",
      fileKey: FILE_KEY,
      isEditor: true
    });
  }

  //无状态用户信息（没有用户信息授权的token时，会用这个接口拿用户信息）
  //start-loading中传入的token既为用户信息授权的token，没有传就认为是无状态用户
  protected override async getUserInfo(): Promise<{
    name: string;
    avatar: string;
  }> {
    return {
      name: USER.name,
      avatar: USER.avatar,
    };
  }
}

```

【接入方前端】初始化继承了PPTGOSdk的子类，并传入初始化参数

```

export const sdkBoxHelper = new SdkBoxHelper(SDK_ORIGIN, {
  backEnable: true, // 是否显示左上角的返回按钮
  onBack() {
    alert("返回页面")
  },
  nameEditable: true, // 是否支持在pptgo sdk中修改文件名称
  updateFileName: (name) => {
    FILE_INFO.name = name;
    return Promise.resolve(name)
  },
  coverEnable: true, // 是否支持封面设置
  setCover: async (cover: Blob) => {
    console.log("set cover", cover);
  },
  thumbnailEnable: true, // 是否支持缩略图设置
  setThumbnail: async (thumb: Blob) => {
    console.log("set thumb", thumb)
  },
  shareEnable: true, // 是否支持显示分享按钮
  onShare() {
    alert("显示分享弹窗")
  },
  async getFileUser() { // 获取关联用户列表，不实现则永远返回空列表，支持分页用户，详细使用
    见 [用户列表分页功能]
    return [
      {
        id: 1,
        name: USER.name,
        avatar: USER.avatar
      },
      {
        id: 2,
        name: "李四",
        avatar: ""
      },
      {
        id: 3,
        name: "王五",
        avatar: ""
      }
    ]
  }
});

```

【接入方前端】通过iframe加载sdk应用页面地址 {Prefix}/app/sdk/

```
<template>
  <iframe
    ref="iframeEle"
    class="sdk-sample--iframe"/>
</template>
```

5. SDK回调申请访问授权

【接入方前端】在SDK触发授权回调后，按接入方业务需要调用接入方服务端接口获取access_token

【接入方后端】根据实际业务场景选择授予什么权限的access_token，调用 [获取文件级access_token](#2. 获取文件级access_token) 接口进行授权

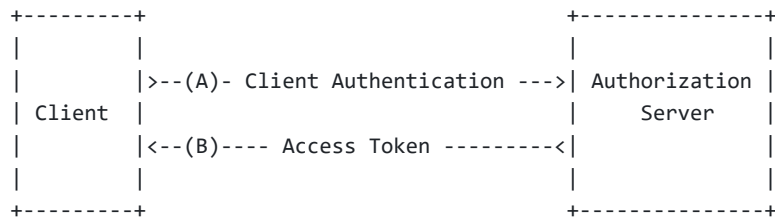
访问授权

PPTGO 面向集成应用提供了一套基于 HTTPS 协议的 OpenAPI ，集成应用可以在获取到访问凭证后，在自身产品内嵌入 PPTGO 编辑器及调用相关的 OpenAPI 完成自制的产品功能。PPTGO SDK采用 **OAuth 2.0 ** 的 [客户端凭证方式](#) 来进行应用身份验证和获取授权。

大致授权流程如下，其中：

Client 表示接入SDK的集成应用（安全考虑，只建议在服务端获取访问凭证）

Authorization Server 表示 PPTGO SDK的授权服务



授权访问凭证access_token

接入方通过 /oauth/token 接口进行应用身份验证后，可获得 PPTGO SDK 颁发的授权访问凭证access_token。

PPTGO SDK 会颁发两类access_token：

- 应用级access_token，可以应用的身份调用相关OpenAPI，可操作归属于应用的所有文档数据，例如创建文件、删除文件等
- 文档级access_token，主要用于集成方颁发给 PPTGO SDK 的编辑器授权使用，可对该文档进行相关的编辑、查看操作

1. 获取client_id 、client_secret

TODO：暂时后台未开发，相关购买流程及收费标准未定。

2. 获取应用级access_token

接口说明

集成方获取应用级 PPTGO SDK的授权访问凭证access_token

请求格式

描述	定义
Path	POST /oauth/token
Content-Type	multipart/form-data

Scope

无

请求头部

详见 [Headers](#Request Headers)。

请求参数

路径参数

无

查询参数

无

请求体

参数名	类型	必选	描述
grant_type	string	是	授权类型，固定client_credentials
scope	string	是	授权操作列表,以空格拼接,all_scopes表示全部
client_id	string	是	应用id
client_secret	string	是	应用密钥

响应体

参数名	类型	描述
access_token	string	访问令牌
expires_in	integer	过期时间，秒
scope	string	授权操作列表，以空格拼接,all_scopes表示全部
token_type	string	令牌类型：bearer

请求示例

```
curl --location --request POST '{Prefix}/oauth/token' \
--form 'grant_type="client_credentials"' \
--form 'scope="all_scopes"' \
--form 'client_id="{client_id}"' \
--form 'client_secret="{client_secret}"'
```

响应示例

```
{  
  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJjaWQiOiJpSW1sdGZLY295WnBuUzRBiwiZGlkIjoiaMTIiLCJpc3MiOiJib2FyZG1peC1zZGsiLCJqdGkiOiI1ZmYyMjY1Ni1lNDQ3LTQ1NmYtYWUyMS02ZWWE3NWI3YTUyYzgiLCJpYXQiOjE2NjE3ODI2MjEsImV4cCI6MTY2Mjc4NDQ4MSwic2NwIjpbImFsbnB9ZyZ29wZXMiXX0.KcDW_eVW0l4jaf0NxU6qyKs7ZDZxAVjuauaWYyskgJk",  
  
  "expires_in": 1799,  
  "scope": "all_scopes",  
  "token_type": "bearer"  
}
```

3. 获取文件级access_token

接口说明

接入方获取文件级access_token后，用于提供给 PPTGO 编辑器SDK鉴权使用。

文件级access_token，接入方可根据自身业务决定授予什么操作权限，在PPTGO SDK编辑器中，目前共分两种角色：编辑者、查看者。授予编辑者权限，可传入scope: "file.edit file.view"; 授予查看者权限，则传入scope: "file.view";

请求格式

描述	定义
Path	POST /oauth/token
Content-Type	multipart/form-data

Scope

无

请求头部

详见 [Headers](#Request Headers)。

请求参数

路径参数

无

查询参数

无

请求体

参数名	类型	必选	描述
grant_type	string	是	授权类型，固定client_credentials
file_key	string	是	文件key
scope	string	是	授权操作列表,以空格拼接 file.view 表示查看权限 file.edit file.view 表示编辑权限
client_id	string	是	应用id
client_secret	string	是	应用密钥
user_id	string	否	当前登录用户标识
nick_name	string	否	当前登录用户昵称
avatar_url	string	否	当前登录用户头像链接

响应体

参数名	类型	描述
access_token	string	访问令牌
expires_in	integer	过期时间，秒
scope	string	授权操作列表，以空格拼接
token_type	string	令牌类型：bearer

请求示例

```
curl --location --request POST '{Prefix}/oauth/token' \
--form 'grant_type="client_credentials"' \
--form 'file_key="{file_key}"' \
--form 'scope="file.edit file.view"' \
--form 'client_id="{client_id}"' \
--form 'client_secret="{client_secret}"' \
--form 'user_id="{user_id}"' \
--form 'nick_name="{nick_name}"' \
--form 'avatar_url="{avatar_url}"'
```

响应示例

```
{  
  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJib2FyZG1peC1zZGsiLCJqdGkiOiI5YmE4YzI3ZC1kZDBkLTRlY2EtOTUzZS0zOGU2ZDMwOWEyZDEiLCJpYXQiOjE2ODEzMzIxNDMImV4cCI6MTY4MTczNTAwMywic2NwIjpbImFsbnB9ZyZ29wZXMiXSwiYXVkIjoick9DS29wbmt2SEg2T2liMXdfX19qQSIsInN1YiI6ImZpbGUuLCJjaWQoiOiJSZWxjVmdXeDRWTfJ1MG11IiwiaWFkIjoimTIiLCJuaWNrIjoimTEExIiwiaXYXZhdGFyIjoid3d3LmJhaWR1LmNvbSIsInVpZCI6IjEifQ.VcjTzBP-EtjrHQ8-_BtZ9Q8qo8HIoMv_cTQGKODQVu0",  
  "expires_in": 1799,  
  "scope": "file.edit file.view",  
  "token_type": "bearer"  
}
```

OpenAPI

创建文件

接口说明

创建一个空白文档。

请求格式

描述	定义
Path	POST /openapi/v1/file

Scope

```
file.create
```

请求头部

详见 [Headers](#Request Headers)。

请求参数

路径参数

无

查询参数

无

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述
data	json	业务数据，详见 [文件信息](#File 文件信息)

请求示例

```
curl --location \  
--request POST '{Prefix}/openapi/v1/file' \  
--header 'Authorization: {token_type} {access_token}'
```

响应示例

```
{  
  "code": 0,  
  "msg": "code-0",  
  "data": {  
    "file_key": "0o1okjVRXrdokufDtaFEJA",  
    "created_at": 1661758256,  
    "updated_at": 1661758256  
  }  
}
```

删除文件

接口说明

删除文件

请求格式

描述	定义
Path	DELETE /openapi/v1/file/{file_key}

Scope

file.delete

请求头部

详见 [Headers](#Request Headers)。

请求参数

路径参数

参数名	类型	必选	描述
file_key	string	是	文件key，唯一键

查询参数

无

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述

请求示例

```
curl --location \
--request DELETE '{Prefix}/openapi/v1/file/{file_key}' \
--header 'Authorization: {token_type} {access_token}'
```

响应示例

```
{
  "code": 0,
  "msg": "code-0"
}
```

创建副本

接口说明

从指定的文件创建一个副本文件

请求格式

描述	定义
Path	POST /openapi/v1/file/{file_key}/copy

Scope

file.copy

请求头部

详见 [Headers](#Request Headers)。

请求参数

路径参数

参数名	类型	必选	描述
file_key	string	是	被拷贝的文件key，唯一键

查询参数

无

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述
data	json	业务数据，详见 [文件信息](#File 文件信息)

请求示例

```
curl --location \
--request POST '{Prefix}/openapi/v1/file/{file_key}/copy' \
--header 'Authorization: {token_type} {access_token}'
```

响应示例

```
{
  "code": 0,
  "msg": "code-0",
  "data": {
    "file_key": "0o1okjVRXrdokufDtaFEJA",
    "created_at": 1661758256,
    "updated_at": 1661758256
  }
}
```

获取aigc签名

接口说明

获取aigc签名

请求格式

描述	定义
Path	POST /openapi/v1/ai/grant

请求头部

详见 [Headers](#Request Headers)。

请求参数

无

路径参数

无

查询参数

无

请求体

参数名	类型	必选	描述
body	string	是	签名body: "{"sequence":"1","timestamp":1692007070}"

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述
data	string	签名

请求示例

```
curl --location \
--request POST '{Host}/openapi/v1/ai/grant' \
--header 'Authorization: {token_type} {access_token}'
--data '{
  "body": "{"sequence\":"1\","timestamp\:1702346565}"
```

响应示例

```
{
  "code": 0,
  "msg": "code-0",
  "data": "08dc645d55ee9b512dc0f480b2a81f87"
}
```

获取 PPTGO 官方模板

接口说明

获取pptgo官方模板列表

请求格式

描述	定义
Path	GET /openapi/v1/cordwood/ppt_theme/list

请求头部

详见 [Headers](#Request Headers)。

请求参数

无

路径参数

无

查询参数

无

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述
data	[]object	业务数据，详见 主题信息

请求示例

```
curl --location \
--request GET '{Host}/openapi/v1/cordwood/ppt_theme/list' \
--header 'Authorization: {token_type} {access_token}'
```

响应示例

```
{
  "code": 0,
  "msg": "code-0",
  "data": [
    {
      "elem_key": "6KH-LONXMzEs5NZdlmJqrg",
      "elem_type": "ppt_template_theme",
      "tags": [
        "scene_1",
        "style_2",
        "blue-01"
      ],
      "thumb_url":
        "https://dev.pptgo.cn/pptsdk/storage/storage_s3/private/ppt_template_theme/6KH-LONXMzEs5NZdlmJqrg/attachment/f70edcdf1f0f3c2cd769fbc1b9eab01/original?
        expire=1729150969&sign=665eQE0t0tpeXnfekEnGVuyW2fioUVCadGGCAPDckxHw"
    }
  ]
}
```

获取 PPTGO 标签列表

接口说明

获取pptgo 标签列表

请求格式

描述	定义
Path	GET /openapi/v1/tag/list

请求头部

详见 [Headers](#Request Headers)。

请求参数

无

路径参数

无

查询参数

参数名	类型	必选	描述
ttype	[]string	是	查询标签类型，theme：主题，scene：场景，style：风格，color：色系，career：职业

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述
data	[]object	业务数据，详见 标签信息

请求示例

```
curl --location \
--request GET '{Host}/openapi/v1/tag/list?ttype[]=theme&ttype[]=color&ttype[]=style' \
--header 'Authorization: {token_type} {access_token}'
```

响应示例


```
{
  "code": 0,
  "msg": "code-0",
  "data": {
    "color": [
      {
        "tkey": "color_1",
        "name": "红色",
        "parent_id": "",
        "icon_url": "",
        "ttype": "color",
        "ext": {
          "extend": {
            "color": 4294000992
          }
        },
        "use": 2,
        "sequence": 0,
        "description": "",
        "created_at": 1727601112,
        "updated_at": 1727601112
      }
    ],
    "style": [
      {
        "tkey": "style_1",
        "name": "扁平简约",
        "parent_id": "",
        "icon_url": "",
        "ttype": "style",
        "ext": {},
        "use": 2,
        "sequence": 1,
        "description": "",
        "created_at": 1727601112,
        "updated_at": 1727601112
      },
      {
        "tkey": "style_2",
        "name": "商务科技",
        "parent_id": "",
        "icon_url": "",
        "ttype": "style",
        "ext": {},
        "use": 2,
        "sequence": 2,
        "description": "",
        "created_at": 1727601112,
        "updated_at": 1727601112
      }
    ],
    "theme": [
```

```
{
  "tkey": "theme_1",
  "name": "深色",
  "parent_id": "",
  "icon_url": "",
  "ttype": "theme",
  "ext": {},
  "use": 2,
  "sequence": 0,
  "description": "",
  "created_at": 1727601112,
  "updated_at": 1727601112
},
{
  "tkey": "theme_2",
  "name": "浅色",
  "parent_id": "",
  "icon_url": "",
  "ttype": "theme",
  "ext": {},
  "use": 2,
  "sequence": 0,
  "description": "",
  "created_at": 1727601112,
  "updated_at": 1727601112
}
]
}
```

获取 PPTGO 推荐md模板列表

接口说明

获取pptgo 推荐md模板列表

请求格式

描述	定义
Path	GET /openapi/v1/recommendation/pre_template

请求头部

详见 [Headers](#Request Headers)。

请求参数

无

路径参数

无

查询参数

无

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述
data	[]object	业务数据，详见 推荐md模板信息

请求示例

```
curl --location \
--request GET '{Host}/openapi/v1/recommendation/pre_template ' \
--header 'Authorization: {token_type} {access_token}'
```

响应示例

```
{
  "code": 0,
  "msg": "code-0",
  "data": [
    {
      "rkey": "lQ-AwtAQGzOhWIby78yRwg",
      "name": "产品经理年中总结报告",
      "file_url":
        "https://sdk.pptgo.cn/pptsdk/storage/storage_s3/pub_url/public/recommendation_file/pre_tem
        plate_1721887371698548768"
    },
    {
      "rkey": "9t50z4fugXQNsCcc8m4p9Q",
      "name": " 打造新媒体广告文案的秘诀",
      "file_url":
        "https://sdk.pptgo.cn/pptsdk/storage/storage_s3/pub_url/public/recommendation_file/pre_tem
        plate_1721887398197349497"
    }
  ]
}
```

上报 PPTGO 官方主题使用记录

接口说明

上报 PPTGO 官方主题使用记录, 列表返回最新的10条

请求格式

描述	定义
Path	POST /openapi/v1/cordwood/ppt_theme/use_history

请求头部

详见 [Headers](#Request Headers)。

请求参数

参数名	类型	描述
elem_key	string	主题key
color_id	string	色系id

路径参数

无

查询参数

无

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述

请求示例

```
curl --location \
--request POST '{Host}/openapi/v1/cordwood/ppt_theme/use_history' \
--d '{"elem_key": "xxxxx", "color_id": "4278190080"}' \
--header 'Authorization: {token_type} {access_token}'
```

响应示例

```
{
  "code": 0,
  "msg": "code-0"
}
```

上报 PPTGO 官方主题收藏

接口说明

上报 PPTGO 官方主题收藏

请求格式

描述	定义
Path	POST /openapi/v1/cordwood/user/collect

请求头部

详见 [Headers](#Request Headers)。

请求参数

参数名	类型	描述
elem_key	string	主题key
collect	bool	true: 收藏， false: 取消收藏

路径参数

无

查询参数

无

请求体

无

响应体

参数名	类型	描述
code	integer	业务返回码
msg	string	业务返回码描述

请求示例

```
curl --location \
--request POST '{Host}/openapi/v1/cordwood/user/collect' \
--d '{"elem_key": "xxxxx", "collect":true}' \
--header 'Authorization: {token_type} {access_token}'
```

响应示例

```
{
  "code": 0,
  "msg": "code-0"
}
```

前端接入

设计简述

pptgo sdk前端本质上是一个无状态的编辑器，本身不保存任何信息、数据。接入方需要向后端签发对应文件的access_token以及传入必要的文件、用户信息，然后sdk前端会根据对应信息与sdk后端构建联系，提供对应文件的查看、编辑等功能。

接入方需要使用iframe加载pptgo sdk提供的页面，为规范并方便接入方与pptgo sdk之间的通信，pptgo sdk前端提供了pptgo-sdk.js的sdk文件，本质上是对于iframe通信的封装，提供了一些规范化的消息通信机制，以及基本的通信校验。

接入流程

1. 在index.html中通过script标签引入pptgo-sdk.js依赖，会在window上挂载PPTGOSdk

```
<script src="{Prefix}/app/pptgo-sdk.js" ></script>
```

2. 继承PPTGOSdk并实现必须的三个抽象方法

```

class SdkBoxHelper extends PPTGOSdk {
  protected override async getAccessToken(): Promise<{
    token: string;
    expiresIn: string
  }> {
    const res = await fetch(`/api/sdk/oauth/token?file_key=${ FILE_KEY }&role=${
FILE_ROLE }`);
    const data = (await res.json()).data;
    return {
      token: data.access_token,
      expiresIn: data.expires_in
    }
  }

  protected override getFileInfo(): Promise<{
    name: string;
    fileKey: string;
    isEditor: boolean;
  }> {
    return Promise.resolve({
      name: "文件",
      fileKey: FILE_KEY,
      isEditor: true
    });
  }

  //无状态用户信息（没有用户信息授权的token时，会用这个接口拿用户信息）
  //start-loading中传入的token既为用户信息授权的token，没有传就认为是无状态用户
  protected override async getUserInfo(): Promise<{
    name: string;
    avatar: string;
    userId: number;
  }> {
    return {
      name: USER.name,
      avatar: USER.avatar,
      userId: USER.userId
    };
  }
}

```

3. 初始化继承了PPTGOSdk的子类，并传入初始化参数

```

// 建议全局单例保存
const sdkBoxHelper = new SdkBoxHelper(SDK_ORIGIN, {
  backEnable: true, // 是否显示左上角的返回按钮
  onBack() {
    alert("返回页面")
  },
  nameEditable: true, // 是否支持在pptgo sdk中修改文件名称
  updateFileName: (name) => {
    FILE_INFO.name = name;
    return Promise.resolve(name)
  },
  coverEnable: true, // 是否支持封面设置
  setCover: async (cover: Blob) => {
    console.log("set cover", cover);
  },
  thumbnailEnable: true, // 是否支持缩略图设置，缩略图配置项见 thumbConfig
  setThumbnail: async (thumb: Blob) => {
    console.log("set thumb", thumb)
  },
  shareEnable: true, // 是否支持显示分享按钮
  onShare() {
    alert("显示分享弹窗")
  },
  async getFileUser() { // 获取关联用户列表，不实现则永远返回空列表，支持分页用户，详细使用见
    [用户列表分页功能]
    return []
  }
});

// 监听PAGE_PREPARED事件，所有消息发送的方法，必须在该事件触发之后才可以发起成功
sdkBoxHelper.on("PAGE_PREPARED", () => {
  ...记录page_prepared状态
})

```

4. 通过iframe加载sdk应用页面地址 {Prefix}/app/sdk/

```

<template>
  <iframe
    ref="iframeEle"
    src="{Prefix}/app/sdk/"
    class="sdk-sample--iframe"/>
</template>
<script>
  const iframeEle = ref();
  onMounted(() => {
    // 绑定iframe标签，该方法可以多次调用用于切换iframe
    sdkBoxHelper.bindIframe(iframeEle.value);
  })
</script>

```

5. 通过注册事件，当sdk页面准备好之后，加载文件

场景一：进入已有文件

```
sdkBoxHelper.on("PAGE_PREPARED", () => {
  sdkBoxHelper.sendMessage("START_LOADING", {
    fileKey: FILE_KEY,
    role: FILE_ROLE, //若传入了用户授权token, 该字段将无效
    token: 'asWqswe*****' //有用户信息授权的token
    thumbConfig: { //缩略图更新配置
      interval: 5 * 60 * 1000, //更新间隔(最小20s, 最大10分钟, 默认5分钟)
      emitOnStart: false, //是否立即触发一次
      mode: "debounce" //缩略图执行方式, 可选参数 "debounce" | "throttle"
    }
  })
})
```

场景二：在用户已经配置好主题和文本内容的情况下，进入编辑页面进行文件内容生成

```
sdkBoxHelper.on("PAGE_PREPARED", () => {
  sdkBoxHelper.sendMessage("START_LOADING", {
    fileKey: FILE_KEY,
    token: 'asWqswe*****', //有用户信息授权的token
    ...其他配置项,
    // 根据配置直接进入文件编辑页生成内容
    createPptOptions: {
      style: 'xxxx'; // 模板key
      text: 'xxx'; // ppt的markdown文本内容
      creationType: "outline" | "markdown"; // 内容生成模式, 'outline': 大纲生成模式,
      // 生成过程中会调用AI进行内容补齐。'markdown': 直接根据传入的文本进行ppt的生成
      aiTextSettingInfo: { // AI高级设置 (非必填项, 仅 creationType 为 outline 模式生效)
        // 语言 建议值 简体中文 | 繁体中文 | 英语 | 日语 | 韩语 | 俄语 | 法语
        // 海外版 建议值 Simplified Chinese | Traditional Chinese | English |
        Japanese | Korean | Russian | French
        useLanguage: "俄语",
        textRichness: "short", // 文本内容丰富度 可选值 "short" | "standard" |
        "rich"
        reportTo: "程序员", // 汇报对象
        tone: "精确、详细的写作, 适合技术文档、手册和指南。强调准确性和清晰性", // 语气
      },
      buildSettingInfo: { // 布局设置 (非必填项)
        isUseTableContents: false, // 生成时是否展示目录页
        isUseSplitPage: true, // 生成时是否展示章节页
        isUseEnding: false, // 生成时是否展示尾页
        splitPageType: "table_contents" // 章节页布局模式 可选值 "single" (单章节
        页) | "table_contents" (目录章节页)
      },
    }
  })
})
```

6. 切换文件需要先退出，再重新加载

```
sdkBoxHelper.sendMessage("STOP_LOADING")
sdkBoxHelper.sendMessage("START_LOADING", {
  ...其他配置项,
  fileKey: FILE_KEY,
  token: 'asWqswe*****' //有用户信息授权的token
})
```

7. 导出PPT

需要在页面触发下载PPT行为

```
sdkBoxHelper.on("LOADING_FILE_SUCCESS", () => {
  sdkBoxHelper.promisePostMessage("DOWNLOAD_PPT").then(() => {
    console.log("下载完成")
  });
})
```

8. 监听PPT内容生成完毕

```
sdkBoxHelper.on("PPT_GENERATE_FINISHED", () => {
  console.log("ppt 内容生成完毕")
})
```

9. 用户列表分页功能

目前分页逻辑采用的是游标查询的方式，使用规则如下

```

export const sdkBoxHelper = new SdkBoxHelper(SDK_ORIGIN, {
  ...其他配置项,
  fileUserMode: "cursor", //指定文件用户模式为 游标模式
  /**
   * 获取用户数据
   * @returns {
   *           userList: { id: number; name: string; avatar: string; }[], //用户列表
   *           nextCursor: number, //下一个游标, 如果当前列表已经是最后一个了, 这个参数需
   *           size: number //当前列表的size
   *         }
   */
  async getFileUser(params: {
    cursor: number; //当前查询的游标(起始游标为 0)
    size: number; //查询的数量
    query?: string | undefined; //关键字过滤
  }) {
    //根据传入的数据进行业务查询
  },
  /**
   * 根据条件查询
   *
   * 目前需要支持的是根据用户id查询列表
   *
   * @returns {
   *   userList: { id: number; name: string; avatar: string; }[], //用户列表数据
   * }
   */
  async getFileUserByCondition(ids: (string | number)[]) {
    //根据传入的userIds返回数据
  }
})

```

10. AI功能对接

1. 注册AI使用签名方法

GET_AI_SIGN 方法在每次使用AI功能接口时, 都会进行一次请求, 用于获取AI接口使用的签名, 该方法会传入下面两个参数, 用于生成签名:

- sequence: 请求序列号, 用于唯一标识一次请求
- timestamp: 请求时间戳, 用于校验请求时间是否过期

签名的使用场景是用于控制用户的AI使用权限, 如果在token中设置了ai.no_auth权限, 则不需要注册这个方法

```

sdkBoxHelper.registPromiseEvent("GET_AI_SIGN",
  async (params: {
    sequence: string; //请求序列号
  }) => {
    const timestamp = Date.now();
    return {
      sign: await ClientApi.getAiSign({ ...params, timestamp }), //返回从后端获取的签
名字符串
      extend: "任意信息" //非必传字段，在调用AI功能时会返回给后端
      timestamp: timestamp; //请求时间戳
    };
  }
)

```

如果单个用户的点数已经用尽，可以在签名接口中返回如下数据，用于限制单用户AI使用

```

sdkBoxHelper.registPromiseEvent("GET_AI_SIGN",
  async (params: {
    sequence: string; //请求序列号
  }) => {
    return {
      errorCode: "right_limit"
    };
  }
)

```

11. 用户点击头像事件透传

```

//开启注入回调
sdkBoxHelper.sendMessage("OPEN_API_INJECT_UI", {
  userCallback: true,
});
//监听回调事件
sdkBoxHelper.on("OPEN_API_EVENT_CALLBACK", (data) => {
  if(data.type === "user") {
    console.log("用户点击头像", data);
  }
})

```

12. 接管文件下载

```

// 监听文件下载事件
sdkBoxHelper.on("DOWNLOAD_FILE", (data: any) => {
  console.log("下载文件", data);
})

// 接管下载行为
// 启用之后将不会触发自动下载，会通过事件的方式下发File对象
sdkBoxHelper.on("PAGE_PREPARED", () => {
  sdkBoxHelper.sendFeatureConfig({
    handleDownload: true // 接管下载行为
  })
});

```

13. 商业化卡点

```

sdkBoxHelper.on("PAGE_PREPARED", () => {
  sdkBoxHelper.sendMessage("START_LOADING", {
    fileKey: "xxxx",
    token,
    fileRight: { // 文件商业权限
      isPro: false, // 是否为专业版
      extend: { // 是否为专业版功能，传false则会显示付费标签，并且点击使用该功能时，会
        触发付费弹窗
          export_ppt: false, // 导出为PPT
          export_pdf: false, // 导出为PDF
          export_img: true // 导出为图片
        }
      }
    });
});

sdkBoxHelper.on("USER_OPERATION", async (data: any) => {
  if (data.operation === "VIP_UPGRADE") {
    console.log("用户触发了升级弹窗")
  }
});

```

d.ts文件

```
declare type SdkEventBoxToApp =
  "INIT_CONFIG"
  | "START_LOADING" // 开始加载文件
  | "STOP_LOADING" // 退出文件
  | "DOWNLOAD_PPT"

declare type SdkEventAppToBox =
  "PAGE_PREPARED" // pptgo应用已准备完成，可以开始进行文件加载
  | "GET_ACCESS_TOKEN" // pptgo请求获取access_token
  | "GET_USER_INFO" // pptgo请求获取当前用户信息
  | "GET_FILE_INFO" // pptgo请求获取当前文件信息
  | "UPDATE_FILE_NAME" // pptgo侧用户修改了文件名称，发送事件请求更新
  | "UPDATE_FILE_COVER" // pptgo侧用户修改了文件封面，发送事件请求更新
  | "UPDATE_FILE_THUMBNAIL" // pptgo侧缩略图变更，发送事件请求更新（缩略图仅返回首页的）
  | "BACK_TO_HOME" // pptgo侧用户点击返回，发送事件请求退出页面
  | "START_SHARE" // pptgo侧用户点击分享按钮，发送事件请求
  | "GET_FILE_USERS" // 部分功能需要关联用户列表，pptgo发送事件请求对应数据
  | "LOADING_FILE_SUCCESS" // 白板文件加载完毕之后触发
  | "LOADING_FILE_FAIL" // 白板文件加载失败触发
  | "LOADING_FILE_START" // 白板文件开始加载时触发
  | "PPT_OUTLINE_UPDATE" // AI对话页，PPT大纲更新时触发
  | "PPT_GENERATE_FINISHED" // 文件编辑页PPT内容生成完毕时触发

type BoxPromiseEventMap = {
  "GET_AI_SIGN": [
    { sequence: string; timestamp: number },
    Promise<{
      sign: string;
    }>
  ];
}

interface IconReplace {
  headerBack?: string; // 回退图标
}

declare abstract class SdkBase {
  public on(event: string, handler: Func);

  public once(event: string, handler: Func);

  public off(event: string, handler: Func);

  public removeAllListener(event: string);

  public emit(event: string, ...args: any[]);
}

declare abstract class PPTGOSdk extends SdkBase {
```

```

    constructor(origin: string, option: SdkBoxOption);

    public bindIframe(iframeEle: HTMLIFrameElement): void;

    public loadFile(fileKey: string): void;

    //功能配置
    public sendFeatureConfig(config: Partial<ComputableFeatureConfig>): void;
    public initFeature(config: Partial<FeatureConfig>): void;

    public override sendMessage(
        key: SdkEventBoxToApp,
        value?: Obj,
        promiseId?: number,
        reply?: boolean
    ): void;

    protected abstract getAccessToken(): Promise<{
        token: string;
        expiresIn: number;
    }>;

    protected abstract getUserInfo(): Promise<{
        name: string;
        avatar: string;
    }>;

    protected abstract getFileInfo(): Promise<{
        name: string;
        fileKey: string;
        isEditor: boolean;
        loginType?: TargetLoginType;
    }>;

    /**
     * promise类型事件的注册
     */
    public registPromiseEvent<T extends keyof BoxPromiseEventMap>(
        name: T,
        func: (args: BoxPromiseEventMap[T][0]) => BoxPromiseEventMap[T][1]
    ): void;

    /**
     * promise类型事件的注销
     */
    public unRegistPromiseEvent<T extends keyof BoxPromiseEventMap>(
        name: T
    ): void;
}

type FILE_ROLE = "editor" | "viewer";

type OptionName =

```

```

    | {
        nameEditable: false;
    }
    | {
        nameEditable: true;
        nameEditMode: "default";
        updateFileName: (name: string) => Promise<string>;
    }
    | {
        nameEditable: true;
        nameEditMode: "control";
        updateFileName: (options: { shapeCount: number }) => void;
    };

```

```

type OptionShare =
    | {
        shareEnable: false;
    }
    | {
        shareEnable: true;
        onShare: () => void;
        onShareWithNode: (nodeId: string) => void;
    };

```

```

type OptionBack =
    | {
        backEnable: false;
    }
    | {
        backEnable: true;
        onBack: () => void;
    };

```

```

type OptionCover =
    | { coverEnable: false }
    | { coverEnable: true; setCover: (cover: Blob) => Promise<void> };

```

```

type OptionThumbnail =
    | { thumbnailEnable: false }
    | { thumbnailEnable: true; setThumbnail: (cover: Blob) => Promise<void> };

```

```

type FileUser = {
    id: number;
    avatar: string;
    name: string;
    mobile?: string;
    email?: string;
};

```

```

type OptionFileUsers = {
    getFileUser?: () => Promise<FileUser[]>;
} | CursorFileUsersOption;

```



```
type CursorFileUsersOption = {
  fileUserMode: "cursor";
  getFileUser: (params: {
    cursor: number;
    size: number;
    query?: string;
  }) => Promise<{
    userList: FileUser[];
    nextCursor?: number;
    size?: number;
  }>;
  getFileUserByCondition: (
    ids: (number | string)[]
  ) => Promise<{ userList: FileUser[] }>;
}

type OptionBase = {
  clientId?: string;
};

export type SdkBoxOption = OptionBase &
  OptionName &
  OptionShare &
  OptionBack &
  OptionCover &
  OptionThumbnail &
  OptionFileUsers;

declare type TargetLoginType = 1 | 2 // 1 编辑, 2 查看
```

附录

Request Headers

参数名	类型	必选	描述
Authorization	string	是	AccessToken 授权访问令牌，用于标识应用和接口鉴权

File 文件信息

参数名	类型	描述
file_key	string	文件key，唯一键，接口中提到的file_key皆为该字段
created_at	int64	文件创建时间，unix时间戳，秒
updated_at	int64	文件更新时间，unix时间戳，秒

PPT主题列表信息

参数名	类型	描述
elem_key	string	主题key，唯一键
elem_type	string	元素类型，ppt_template_theme:主题
thumb_url	string	封面url
tags	[]string	标签列表

主题标签信息

参数名	类型	描述
tkey	string	标签唯一key
name	string	标签名字
ttype	string	类型
ext	object	标签扩展配置
sequence	int64	排序

主题标签ext信息

参数名	类型	描述
name	string	标签名字
extend	json	额外配置

多语言信息

参数名	类型	描述
text	string	默认语言名称
multilingual	map[string]string	对应多语言，zh, en

推荐md模板信息

参数名	类型	描述
rkey	string	唯一key
name	string	名字
file_url	string	下载地址

scope列表

scope	说明
file.view	文档查看相关权限
file.edit	文档操作相关权限
file.copy	创建文档副本权限
file.create	新建文档
file.delete	删除文档

修订记录

时间	说明
2024-08-23	初版SDK文档编写
2024-10-10	新增ppt主题和标签查询接口
2024-11-05	新增模板md文件推荐列表查询接口
2024-11-12	新增AI对话功能使用场景
2024-12-17	新增用户点击头像事件透传 接管文件下载